

AN EXPERIMENTAL INTRODUCTION TO THE VECTOR SEQUENCE

J. Collins
CollinsJD23@gcc.edu

*An intelligent heart acquires knowledge, and the ear of the wise seeks knowledge.
Proverbs 18:15*

Abstract

Here, we explore how common number theory functions act within a special environment, the vector sequence. Specifically, we study the General Hailstone, Aliquot, Euler-Totient, and Sum-of-Divisors functions (HATS). We are most interested in the behavior characteristics of HATS in this environment and use the tools in this *GitHub Repository* to do so.

The vector sequence is a recursive object, generated from an initial vector and vector operator, T . This operator houses parameters $S : \mathbb{Z} \rightarrow \mathbb{Z}$ and k . Properties of the vector sequences are shown to depend on S and k and include parity, finite memory, chaotic-looking behavior, micro-structures, macro-structures, and cycles.

Other function-specific or computational parameters were found to change results and, often times, cause unique patterns. No singular model was found that predicted the behavior of all vector sequences; however, some models were found to describe certain vector sequences for particular HATS functions.

Many niche areas for further studies are outlined throughout this paper, while unifying areas for further studies are listed in Section 7. Overall, much is outlined and much is still left to be explained.

1. Introduction

All four of the HATS functions were independently formalized between the 18th and 20th centuries and have been areas of significant study ever since. Despite substantial exploration, the behavior of these functions remains nontrivial, irregular, and, in some cases, chaotic-looking which was the main motivation for implementing them in the vector sequence framework. Additional functions like $Rad(n)$ and the Divisor Product Function, $P(n)$, were considered but not implemented due to their runtime and spacetime complexities. To see just how infeasible functions like these are for us to study, we outline a quick Big-O analysis.

Assuming ideal RAM, multiplications have constant runtime complexity. Since $Rad(n)$ and $P(n)$ require trial-factorization of n , $Rad(n) = P(n) = O(\sqrt{n})$. Also, assuming no caching, our spacetime complexity is $O(1)$. As we will see in later sections, often computations require $n \sim 10^{25}$ and so we quickly reach computations that require years to complete. Thus, we must trade off runtime complexity for spacetime complexity via caching. To quantify the effects of this, for spacetime complexity, first note that $P(n) > Rad(n) = O(n)$ where n requires $\lceil \log_2 n \rceil$ bits to store. Thus, to achieve an acceptable run time complexity, without loss of generality, we must cache $O(\lceil \log_2(n) \rceil) = O(\log_2(n))$ bits for each n . Therefore, the spacetime complexity of $Rad(n)$ and $P(n)$ is $O(n \cdot \log_2(n))$ bits. Because for common, large computations $n \sim 10^{25}$, an impactful cache should store at least 10% of the total computations; however, 10% of 10^{25} is 10^{24} so the cache must store at least 10^{24} $Rad(n)$ or $P(n)$ values. Therefore, each cache must be on the order of $10^{24} \cdot \log_2 10^{24} \approx 7.97 \cdot 10^{25}$ bits or roughly 10^{16} GB. This is not feasible, so, we do not consider functions like $Rad(n)$ or $P(n)$ here.

The first function we explore in Section 3 is the General Hailstone function,

$$h_q(n) = \begin{cases} qn + 1 & \text{if } n \equiv_2 1 \\ n/2 & \text{if } n \equiv_2 0 \end{cases}.$$

This was introduced by Lothar Collatz in 1937 and is an example of a simple function with non-trivial behavior [4]. Furthermore, this function is the basis for the Collatz Conjecture, which states for the case of $q = 3$, for all $n \in \mathbb{N}$ there exists some $m \in \mathbb{N}$, such that m compositions of the Hailstone function on n equals 1 [4]. This conjecture aligns with our studies of the vector sequence, but is not something we directly focus on. We conclude that vector sequences implementing this function seem to have many opportunities for further exploration.

Then, in Section 4, we look at the Aliquot or Sum of Proper Divisors function which has its origins in the study of perfect and amicable numbers and is defined by,

$$s(n) = \sum_{d|n, d \neq n} d.$$

Like the Collatz Conjecture for the $q = 3$ case of the General Hailstone function, the Catalan-Dickson Conjecture posits that for all $n \in \mathbb{N}$ there exists an $m \in \mathbb{N}$, such that n compositions of the Aliquot function on n equals 0 or enters a finite cycle [1]. This conjecture was introduced in 1888 and then strengthened in 1913 by Eugène Catalan and Leonard Dickson respectively. Like the Collatz Conjecture, the Catalan-Dickson Conjecture aligns with what we study but is not a direct focus of ours. We conclude that vector sequences implementing this function seem to have very limited opportunities for further exploration.

Next, in Section 5, we look at the Euler-Totient function, introduced by Leonard Euler in 1763 [2], defined as,

$$\phi(n) = n \cdot \prod_{p|n} (1 - 1/p).$$

In the vector sequence, this function behaves more regularly than the previous two functions. We suspect this may be because it is known that a sufficient number of compositions of the Euler-Totient function on any integer equals 1 [3]. We study this function because of this more regular and predictable behavior, since it could make for a good comparison. We concluded that vector sequences implementing this function seem to have minimal opportunities for further exploration.

In Section 6 we look at the last HATS function, the Sum of Divisors function, which emerged from Greek number theory and is defined as,

$$\sigma(n) = \sum_{d|n} d.$$

This function has no associated formal open conjecture, but it exhibits irregular and chaotic-looking behavior as well as being ubiquitous in number theory, so it is explored. We concluded that vector sequences implementing this function seem to have multiple opportunities for further exploration.

Lastly, Section 7 compares the results of the different functions, Section 8 covers our computational challenges, and limitations, and Section 9 lists our references.

2. Definitions

Definition 1. Let $v^{(n)}$ be an integer vector of n -dimensions.

Definition 2. Let $n, N \in \mathbb{Z}$, $T : \mathbb{N}^n \rightarrow \mathbb{N}^{n+1}$, and $S : \mathbb{Z} \rightarrow \mathbb{Z}$. We define a *vector sequence* to be $V = (v_0^{(1)}, v_1^{(2)}, \dots, v_N^{(N+1)})$ and for some $x_0 \in [1, x_{end}]$,

$$\begin{aligned} v_0^{(1)} &= (x_0), \\ v_{n+1}^{(n+2)} &= T(v_n^{(n+1)}) = (\lfloor x_0/k \rfloor, \lfloor x_1/k \rfloor, \dots, \lfloor x_n/k \rfloor, S(\sum_i \lfloor x_i/k \rfloor)). \end{aligned}$$

Note, in this paper occasionally $x_0 \in [2, x_{end}]$, not $[1, x_{end}]$. Experimental results remain similar enough regardless of the minimum x_0 value, but theoretically the minimum x_0 value should be 1.

Also, for this paper, N and x_{end} are computational bounds. Since we cannot let the vector sequence contain infinite elements, N bounds our vector sequence length and x_{end} bounds our range of initial values x_0 ; however, theoretically, a vector sequence lets $N, x_{end} \rightarrow \infty$.

Definition 3. We say V is a *zeroing vector sequence* if V contains an n -dimensional zero vector and write $Z_{\#}$ for the number of zeroing vector sequences in some context.

Definition 4. We say V is a *cycling vector sequence* if there exists $l > 0$ such that $v_p = v_{p+l}$ and $v_p, v_{p+l} \in V$ and write $C_{\#}$ for the number of cycling vector sequences in some context.

Definition 5. We say V is a *growing vector sequence* if V does not contain an n -dimensional zero vector and write $G_{\#}$ for the number of growing vector sequences in some context.

Experimentally for all of our growing vector sequences, $i \rightarrow \infty \implies \|v_i\| \rightarrow \infty$, but by our definition this is not required for V to be a growing vector sequence. All that is required is that v_i does not make V a cycling vector sequence. Thus, for example, there could exist a case where $i \rightarrow \infty$, $\|v_i\| \rightarrow C$ for some $C \in \mathbb{R}$. Our programs are not coded to find these cases, but these cases are an area for further study.

Definition 6. We write $\Sigma_{\#}$ for the total number of growing, zeroing, and cycling vector sequences in some context.

By Definition 6, $\Sigma_{\#} = C_{\#} + G_{\#} + Z_{\#}$. For all S , there exist k ranges that result in $Z_{\#} = 0$ or $G_{\#} = 0$ and so it is important to note that if $Z_{\#} = 0$, then $G_{\#} = \Sigma_{\#} - C_{\#}$, and if $G_{\#} = 0$, then $Z_{\#} = \Sigma_{\#} - C_{\#}$.

With these definitions and tools from this **GitHub Repository** we will explore the vector sequences implementing HATS. This is a computationally heavy exploration where we push our code to its limitations. Thus, unfortunately, we were unable to achieve ubiquitous parameter values for all tests. Occasionally, this is intentional so that we can compare data and show how changing parameters effects results. Lastly, all graphs in this paper plot k versus $Z_{\#}$, $G_{\#}$, and $C_{\#}$ and so we will refer to these as ZGC graphs.

3. General Hailstone Function

In this section we let $S(n)$ be the General Hailstone function,

$$S(n) = h_q(n) = \begin{cases} qn + 1 & \text{if } n \equiv_2 1 \\ n/2 & \text{if } n \equiv_2 0 \end{cases}.$$

From experimentation, we determined that the ZGC graphs in this section depend on both q and x_{end} . Since each ZGC graph is, by definition, 2-dimensional, we do not represent these extra 2 dimensions on the same graph. This would create a 4 dimensional mess. Instead, we define two 3-dimensional graphs: q -ZGC graphs and x_{end} -ZGC graphs. The dimensions of the q -ZGC graphs are k , $(G_{\#}, C_{\#}, Z_{\#})$, and q and the dimensions of the x_{end} -ZGC graphs are k , $(G_{\#}, C_{\#}, Z_{\#})$, and x_{end} . These graphs are simply multiple ZGC graphs in which we vary q or x_{end} . Intuitive examples can be seen in the animations in the GitHub repository.

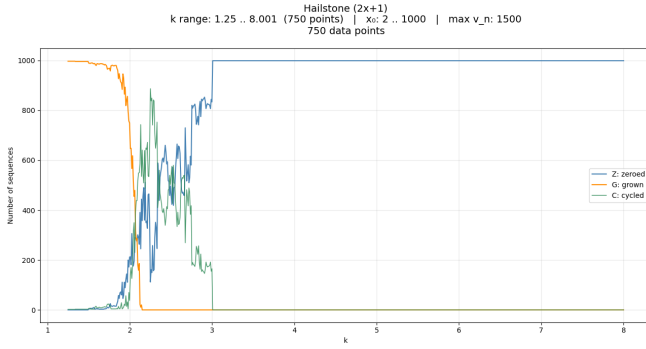
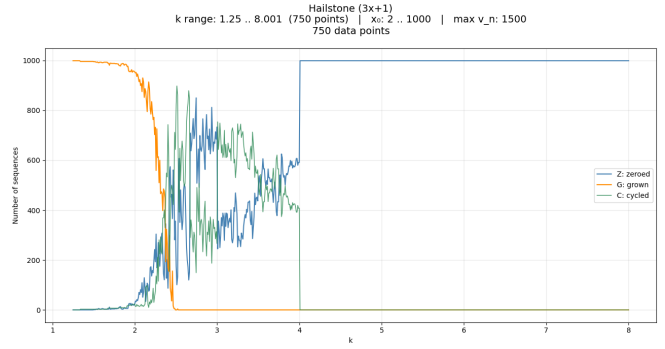
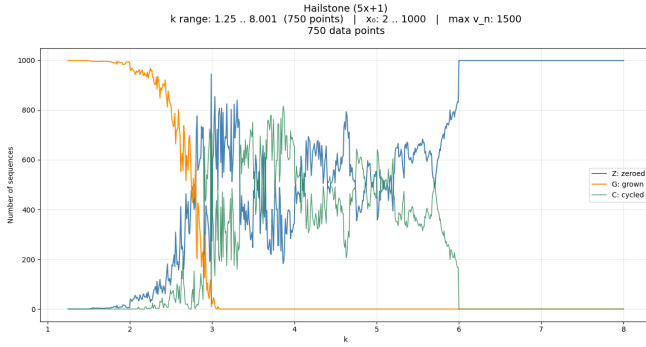
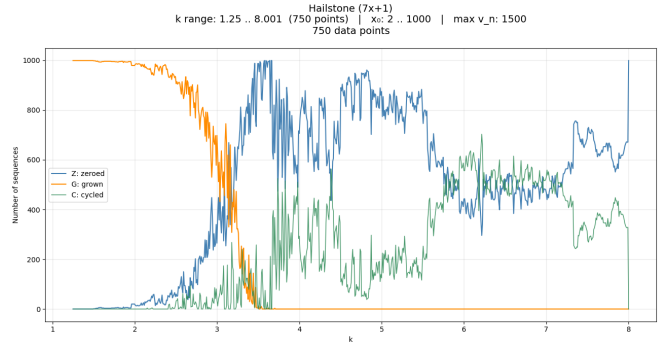
In addition to q -ZGC graphs and x_{end} -ZGC graphs it is useful to subdivide all ZGC graphs into three categories depending on their values q and x_{end} . All ZGC graphs begin with a $G_{\#}$ -dominated region and end with a $Z_{\#} = \Sigma_{\#}$ region, but are separated by a $Z_{\#}$ and $C_{\#}$ region that is either (1) entirely chaotic-looking, (2) mostly chaotic-looking, or (3) somewhat chaotic-looking. To quantify the differences, we can say that either a section is chaotic-looking or semi-linear where semi-linear is defined as being where $Z_{\#}$ contains small structures forming a larger piecewise linear graph. For reference, common examples of what would be called semi-linear include finite square or triangular Fourier waves. We say that Type 1 graphs contain semi-linear regions spanning less than 2% of the total k range, Type 2 graphs contain semi-linear regions spanning 2% – 25% of the total k range, and Type 3 graphs contain semi-linear regions spanning $> 25\%$ of the total k range. Therefore, Type 1 graphs are entirely chaotic-looking, Type 2 graphs are mostly chaotic-looking, and Type 3 graphs are somewhat chaotic-looking. We want to emphasize that there is no sharp distinction between graph types; they morph into each other. So, even with Type 1 graphs, Type 2 and Type 3 structures are present.

Next, we define a constant, x_{end}/q^2 . This value proved to be a reliable indicator for the type of ZGC graph. If we let k range from 1 to $q+1$, let the number of data points be infinite, and let $N \rightarrow \infty$, then the Type 1 graphs satisfy $x_{end}/q^2 \gtrsim 0.8$, Type 2, $0.8 \gtrsim x_{end}/q^2 \gtrsim 0.1$, and Type 3, $x_{end}/q^2 \lesssim 0.1$. This implies and explains why semi-linear regions only emerge as q increases and why chaotic-looking regions only emerge as x_{end} increases. In the former, x_{end}/q^2 strictly decreases and, in the latter, x_{end}/q^2 strictly increases. This behavior is visualized in the q -ZGC and x_{end} -ZGC graph animations in the GitHub repository.

It should be noted that for sufficiently small x_{end} , all ZGC graphs lack detail due to the lack of data, not the value x_{end}/q^2 . We recommend keeping $x_{end} > 15$ to keep enough detail to show the behavior of the ZGC graphs.

3.1. Type 1 ZGC Graphs

Due to the stipulation $x_{end} > 15$, the $q = 3$ Hailstone function related to the Collatz Conjecture will have a x_{end}/q^2 value of at least $15/3^2 = 1.4$. Thus, it will always be a Type 1 ZGC graph. Similarly, all $q < 19$ ZGC graphs will be Type 1. With that in mind, we expect that these low q value graphs are entirely chaotic-looking which can be seen by the following examples. For ease of comparison, each of the following ZGC graphs ranges from $k = 1.25$ to $k = 8$.

(a) $q = 2, x_{end} = 1000$ (b) $q = 3, x_{end} = 1000$ (a) $q = 5, x_{end} = 1000$ (b) $q = 7, x_{end} = 1000$

Notice that the $q = 2$ case is non-trivial. Because k is a decay parameter, we do not need the General Hailstone function to involve decay. Therefore, even values of q result in non-trivial behavior.

For all q , moving in the positive k direction, we see the expected initial region dominated by $G_{\#}$, then the Type 1, entirely chaotic-looking region, and then finally a region dominated by $Z_{\#}$. The predictability of this final region leads to our first experimental conjecture.

Conjecture 1. If $q \in \mathbb{N}$ and $k > q + 1$, then $Z_{\#} = \Sigma_{\#}$.

In the chaotic region of these ZGC graphs we see relatively large changes in $Z_{\#}$ and $C_{\#}$. Since some occur at familiar numbers, we list the k values at which these large changes occur for the four graphs above.

For $q = 2$, $\Delta Z_{\#} > 0.3 \cdot \Sigma_{\#}$ (i.e., $Z_{\#}$ has a large change) at $k \approx 2.333$ and 2.666 . We suspect that these are the rational numbers $k = 7/3$ and $k = 8/3$.

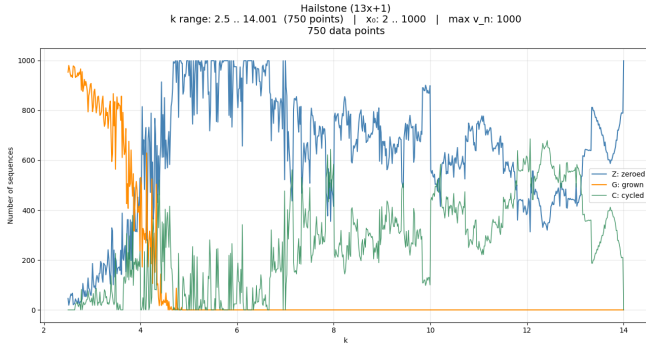
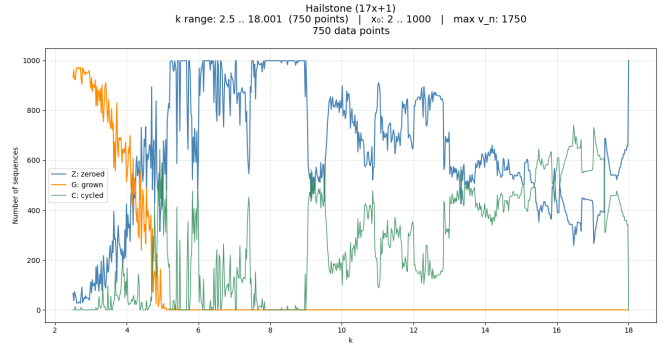
For $q = 3$, $\Delta Z_{\#} > 0.3 \cdot \Sigma_{\#}$ at the rational values $k = 2.5, 2.666..., 2.777..., 3$, and 4 and at possibly irrational $k \approx 2.411, 2.435, 2.444, 2.4690026954$ and 2.477 .

For $q = 5$, $\Delta Z_{\#} > 0.3 \cdot \Sigma_{\#}$ at $k \approx 2.822, 2.865, 2.898, 2.964, 2.969, 2.988, 2.993, 3.245$, and 3.654 . We have not determined which of these are rational/irrational.

For $q = 7$, $\Delta Z_{\#} > 0.3 \cdot \Sigma_{\#}$ at $k \approx 3.199, 3.666, 3.821, 3.835$ and 8 . Besides $k = 8$ we have not determined which of these are rational/irrational.

Looking at $\Delta Z_{\#}$ greater than $0.2 \cdot \Sigma_{\#}$ or even $0.1 \cdot \Sigma_{\#}$ may lead to interesting results; however, we found that it introduced something similar to noise. We suspect that the number and location of these large changes in $Z_{\#}$ may be related to the value of x_{end}/q^2 , but this remains an area for further studies.

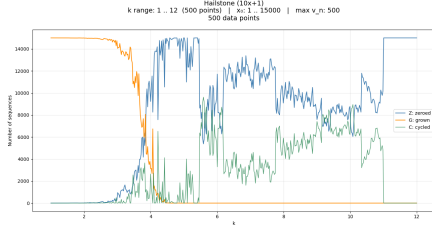
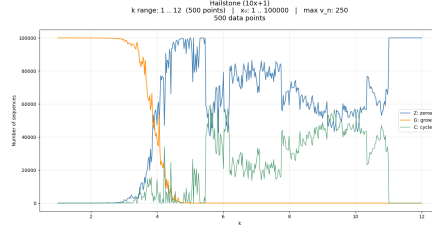
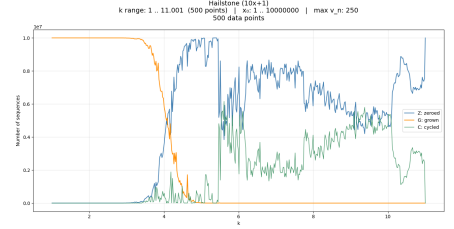
An example of the aforementioned Type 2 and Type 3 behavior present in Type 1 graphs is the sharp change in $Z_{\#}$ (and so $C_{\#}$) at $k \approx (q + 1)/2$. Some additional Type 1 examples showing this are below.

(a) $q = 13, x_{end} = 1000$ (b) $q = 17, x_{end} = 1000$

Therefore, we pose the following experimental conjecture.

Conjecture 2. For all ZGC graphs, at $k \approx (q+1)/2$, $\Delta Z_{\#} << 0$.

It also appears that macro-structures may form in the semi-linear region, for $x_{end}/q^2 \gtrsim 150$. Our hardware struggled to compute singular ZGC graphs let alone a q -ZGC or x_{end} -ZGC graph animation for high enough values of x_{end}/q^2 to explore this properly, so the possibility of new macro-structures existing for such extreme Type 1 graph cases is an area for further study. Below are some graphs that we were able to create showing this behavior in this extreme realm.

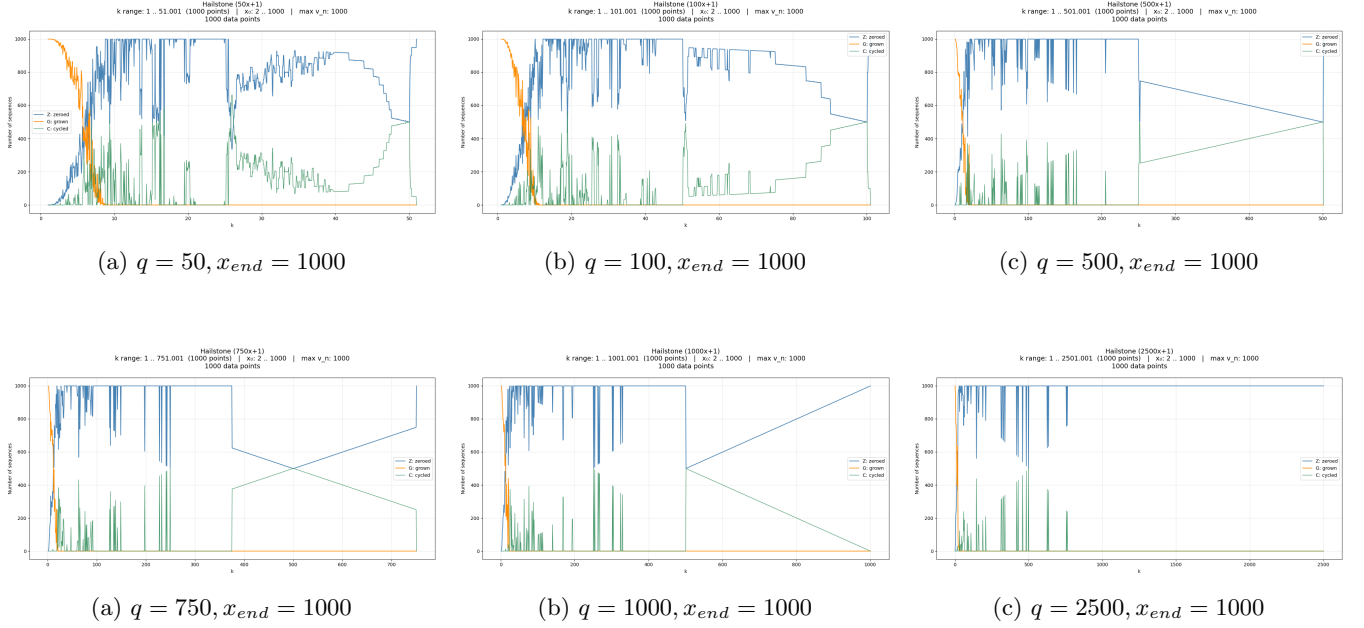
(a) $q = 10, x_{end} = 15,000$ (b) $q = 10, x_{end} = 100,000$ (c) $q = 10, x_{end} = 10,000,000$

3.2. Type 2 and Type 3 ZGC Graphs

The large change in $Z_{\#}$ in Conjecture 2 becomes the divide between the fully chaotic-looking and semi-linear regions in Type 2 and Type 3 graphs. This change in $Z_{\#}$ at $k \approx (q+1)/2$ could be seen to some degree in all graphs, so we posit the following conjecture.

Conjecture 3. For all ZGC graphs, the region $k \in [0, (q+1)/2)$ is chaotic-looking and the region $k \in ((q+1)/2, q+1]$ is semi-linear for Type 2 and Type 3 graphs.

While Type 1 ZGC graphs look chaotic, Type 2 and Type 3 graphs have regions of visual structure. Furthermore, because this structure is cohesive across q -ZGC and x_{end} -ZGC graphs, MP4 animations of relatively large range q -ZGC and x_{end} -ZGC graphs are provided in the GitHub repository. Let D be the number of data points for each ZGC graph in the q -ZGC graph and the animation parameters for the q -ZGC graph are $q_{start} = 2$, $q_{end} = 2500$, $k_{start} = 1$, $k_{end} = q + 1.001$, $x_{start} = 1$, $x_{end} = 1000$, $N = 1000$, and $D = 1000$. Similarly, the animation parameters for the x_{end} -ZGC graph are $q = 100$, $x_{start} = 1$, $(x_{end})_{start} = 1$, $(x_{end})_{end} = 6400$, $k_{start} = 1$, $k_{end} = 102.001$, $N = 500$, and $D = 1000$. Folders containing CSV file caches for each animation's computations are also available in the repository. The following graphs summarize some of the data.



From the listed q and x_{end} values above, we calculate and list the q and x_{end}/q^2 values for each graph as the following tuples of the form $(q, x_{end}/q^2)$, omitting x_{end} since it is always 1000: $(50, 0.4)$, $(100, 0.1)$, $(500, 0.004)$, $(750, 0.0017\dots)$, $(1000, 0.001)$, and $(2500, 0.00016)$. Thus, by our relationship between x_{end}/q^2 and the type of ZGC graph, we see that the $q = 50$ and $q = 100$ cases are Type 2 graphs and the rest are Type 3. The visual characteristics of each graph support this. For the $q = 50$ and $q = 100$ cases we see some chaotic-looking behavior in the semi-linear region (i.e. $k > (q + 1)/2$) and then for $q \geq 500$ there is no chaotic-looking behavior in this region.

The semi-linear behavior has enough structure that we can represent an approximate, ideal version of $Z_{\#}$ (and therefore trivially $C_{\#}$) as a piecewise semi-linear function. To do so, we must determine the relationship between q , x_{end} , $\Sigma_{\#}$, and k and the slopes, y-intercepts, number of line segments, and domain for each line segment for each ZGC graph. Unless stated otherwise, when we say a line segment, we are referring to a $Z_{\#}$ line segment, not a $C_{\#}$ line segment.

From the graphs, we first note that each line segment in the semi-linear region has a different slope. Thus, we apply the elementary rise over run formula for slope to the computed data points for each line segment. In this way, we can create a table of slopes for different pairs (q, x_{end}) . We found that for any $q, a \in \mathbb{N}$, looking at the values where $x_{end} = a \cdot q$ showed relationships most clearly.

The sequences listed in the following table are the approximate slopes of the line segments of the ZGC graph for the given pair (q, x_{end}) . Confirmation bias could have effected these results since noise was present, but once a pattern emerged we sifted through it according to that pattern. The sequences of slopes are listed from the smallest to the largest k value line segments. Empty cells are where the ZGC graph becomes too close to a Type 1 graph that the semi-linear region cannot be accurately represented by the following table. This is subjective though, so review of our edge cases is encouraged.

$q \setminus x_{end}$	$q/2$	q	$2q$	$3q$	$4q$	$5q$	$6q$	$7q$	$8q$
10	0	1	-1	-1, 2					
25	0	1	-1	-1, 2	-1, -2	-1, -2, 3			
50	0	1	-1	-1, 2	-1, -2	-1, -2, 3	-1, -2, -3		
100	0	1	-1	-1, 2	-1, -2	-1, -2, 3	-1, -2, -3	-1, -2, -3, 4	-1, -2, -3, -4
250	0	1	-1	-1, 2	-1, -2	-1, -2, 3	-1, -2, -3	-1, -2, -3, 4	-1, -2, -3, -4

Table 1: Slopes of line segments in the semi-linear region of Type 2 and Type 3 ZGC graphs for pairs (q, x_{end}) .

The general case that we found for the result of $x_{end} = q/2$ is that for all $x_{end} < q$, there exists only one line segment in the semi-linear region and this line segment has a slope of 0.

Although this table is not very large, it encapsulates the pattern for all the values $x_{end} = a \cdot q$ that we saw.

Between the values $x_{end} = a \cdot q$ and $x_{end} = (a + 1) \cdot q$, there exists an aesthetic transition that can be clearly seen in the q -ZGC and x_{end} -ZGC animations. However, unlike the cases where $x_{end} = a \cdot q$ or equivalently, $q|x_{end}$, this transition appears to be best described by an algorithm, not by a function. Therefore, we will define $Z_{\#}(q, x_{end}, \Sigma_{\#}, k)$ as a piecewise function applicable only when $x_{end} = a \cdot q$ and so the cases where $q \nmid x_{end}$ are a place for future study.

From our data, if extended, all line segments pass through the point $(0, \Sigma_{\#})$. Therefore, all line segments have a y-intercept of $\Sigma_{\#}$.

Also, from our data, the transition from the n th to the $(n + 1)$ th line segment occurs at the value $k = q/2 + q/2 \cdot n/(n + 1) = q/2 \cdot (1 + n/(n + 1))$. This experimental result forms the basis for our domain cases in the piecewise function.

Lastly, for a set of initial parameters, we need to know the number of line segments, $L_{\#}$, in the ZGC graph. From Table 1, we can see that a new line segment is created (i.e. a discontinuous jump occurs) when a is odd. Thus, for $l \in \mathbb{Z}$ a new line segment is created when $x_{end} = (2l + 1) \cdot q$. Notice then that from our table $L_{\#} = l + 1$. Therefore, $L_{\#} = l + 1 = (x_{end} + q)/2q$. Again, this only holds when x_{end} is an odd multiple of q ; generally, $L_{\#} = \lfloor (x_{end} + q)/2q \rfloor$.

Now, with the slope, y-intercept, domains for each case, and number of cases defined in terms of our initial parameters, when $q|x_{end}$, we describe the semi-linear region of a ZGC graph as,

$$Z_{\#}(q, x_{end}, \Sigma_{\#}, k) = \begin{cases} \Sigma_{\#} - k & \text{if } q/2(1 + 0/1) < k < q/2 \cdot (1 + 1/2) \\ \Sigma_{\#} - 2k & \text{if } q/2(1 + 1/2) < k < q/2 \cdot (1 + 2/3) \\ \Sigma_{\#} - 3k & \text{if } q/2(1 + 2/3) < k < q/2 \cdot (1 + 3/4) \\ \vdots & \\ \Sigma_{\#} - L_{\#}k & \text{if } q/2(1 + (L_{\#} - 1)/L_{\#}) < k < q/2 \cdot (1 + L_{\#}/(L_{\#} + 1)) \end{cases} = \Sigma_{\#} - k \cdot \left\lfloor \frac{q}{2(q - k)} \right\rfloor.$$

Since $G_{\#} = 0$, in this region $C_{\#} = \Sigma_{\#} - Z_{\#}$, so,

$$C_{\#}(q, x_{end}, k) = \begin{cases} k & \text{if } q/2(1 + 0/1) < k < q/2 \cdot (1 + 1/2) \\ 2k & \text{if } q/2(1 + 1/2) < k < q/2 \cdot (1 + 2/3) \\ 3k & \text{if } q/2(1 + 2/3) < k < q/2 \cdot (1 + 3/4) \\ \vdots & \\ L_{\#}k & \text{if } q/2(1 + (L_{\#} - 1)/L_{\#}) < k < q/2 \cdot (1 + L_{\#}/(L_{\#} + 1)) \end{cases} = k \cdot \left\lfloor \frac{q}{2(q - k)} \right\rfloor.$$

Notice that the domains of the piecewise solutions are $(q/2, q/2 \cdot (2L_{\#} + 1)/(L_{\#} + 1))$ while the domains of the simpler closed-form solutions are $(-\infty, q) \cup (q, \infty)$. Thus, the equality only holds for values of k in both domains.

Since the piecewise solutions include x_{end} information that the simple closed-form solutions do not, we can conclude that x_{end} does not impact the behavior of the vector sequence for k in the semi-linear region; instead, it *defines* where the semi-linear region is.

Also, now we can better understand why the semi-linear region appears/disappears for the values it does. Notice that in the x_{end} -ZGC graph animation we can see that as the semi-linear region is developing, the beginning of it starts to form chaotic-looking spikes and that these changes are precipitated by increasing x_{end} . Therefore, the development of the semi-linear region must only result from increasing x_{end} . Since the values of $L_{\#}$ and x_{end}/q^2 depend on x_{end} , the result of changing these values must be causing this change in the graphs. For the semi-linear region to develop more steps and appear to smooth out, $L_{\#}$ must increase and since $L_{\#} \propto x_{end}$, x_{end} must increase. Therefore, assuming q is constant, by increasing x_{end} , x_{end}/q^2 also increases; however, as x_{end}/q^2 increases, the amount of chaotic-looking behavior also increases. Thus, as the semi-linear region develops more line segments, it also becomes more chaotic-looking. We can then say that for any q there exists a subjective, optimal x_{end} value such that the semi-linear region is highly developed, but not too overrun by chaotic-looking behavior. We found that for a given q , this value of x_{end} can be found when $x_{end}/q^2 \approx 0.5$. Again, this is subjective, but is important as it is a quantitative marker for a transition from order to chaos in the semi-linear region.

While we focused on the $k \geq q/2$, semi-linear region of Type 2 and Type 3 ZGC graphs, it is evident that there exists a "shadow structure" in the $k \leq q/2$, chaotic-looking region of these same graphs. Here, the tips of some of the $Z_{\#}$ and $C_{\#}$ spikes form linear equations (examples can be seen in the graphs at the beginning of this section). This phenomenon seems tied to studying the encroaching, chaotic-looking behavior in the semi-linear region since

this could be an example of the reverse effect, where semi-linear behavior is encroaching into the chaotic-looking region. This is also an area for further study.

4. Aliquot Function

In this section we let $S(n) = s(n) = \sum_{d|n, d \neq n} d$ which is to say $S(n)$ equals the sum of all the proper divisors of positive integer n excluding n itself. Therefore, we let $S(n)$ equal the Aliquot function. Below is the resulting ZGC graph.

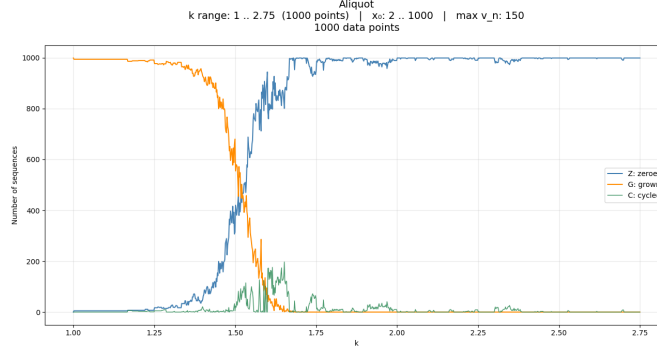


Figure 7: Time to compute: 56308.8s

Although the ZGC graph above has a relatively low N of 150, our program took 15.6 hours to run. Since runtime increases exponentially with N , we began to hit our upper limit for runtime with $N = 150$. To be confident in our results, we used *vector_sequence_GUI.py* to compute a higher N for a few k values between 1.25 and 1.625 and compared those with $N = 150$. We noticed negligible differences in data points above $N = 350$ and even the ZGC graph for $N = 150$ was not far off from these values as the difference in $G_{\#}$, $C_{\#}$, and $Z_{\#}$ between $N = 150$ and $N = 350$ was at most ± 15 .

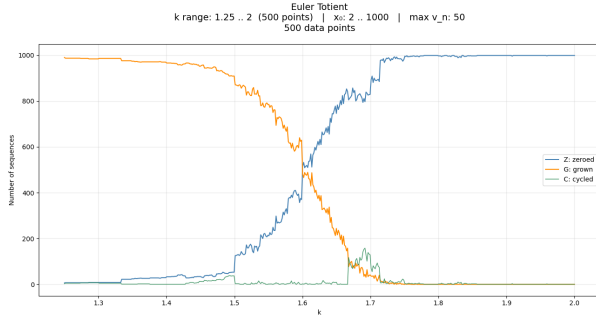
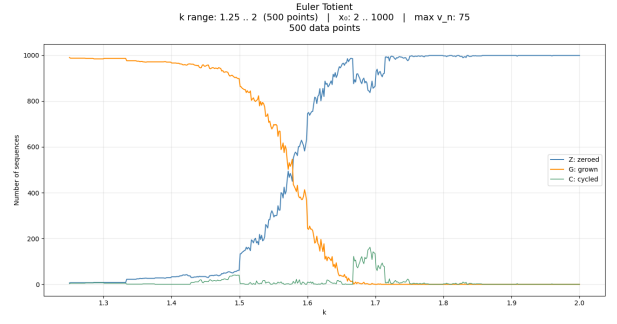
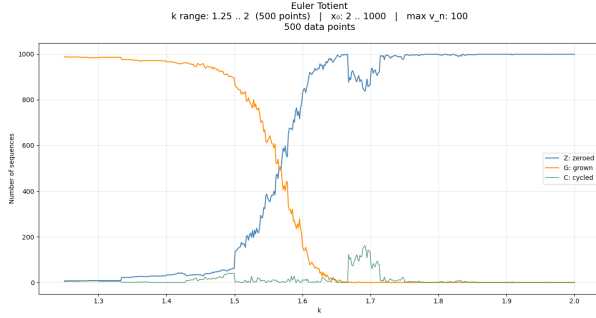
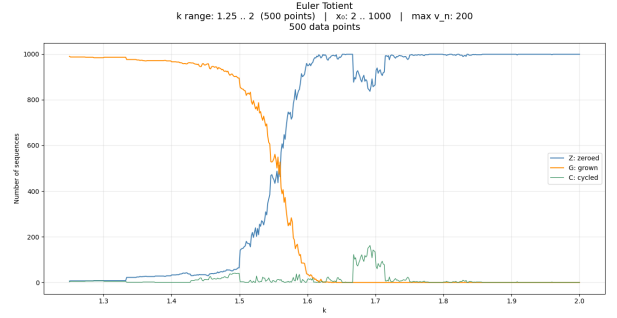
There are a few main features of Figure 7. Firstly, it is evident that $Z_{\#}$ and $G_{\#}$ are largely symmetric and dominate $\Sigma_{\#}$. Also, only between $k = 1.5$ and $k = 1.75$ is there a noticeable number of cycling vector sequences. Furthermore, the expected tail region where $Z_{\#} \approx \Sigma_{\#}$ and $C_{\#} \approx 0$ extends to $k \approx 2.75$, after which we believe $Z_{\#} = \Sigma_{\#}$ and $C_{\#} = G_{\#} = 0$.

Overall, this function proved computationally expensive, and seems to lack areas for further study.

5. Euler-Totient Function

In this section, we let $S(n) = \phi(n) = \prod_{p|n} (1 - 1/p)$ which is to say $S(n)$ counts the number of positive integers up to some positive integer n that are co-prime to n . Therefore, we let $S(n)$ equal the Euler-Totient Function.

In this section, N changes the behavior of the ZGC graph up through $N = 400$. The following ZGC graphs show this change where the $N = 50$, $N = 75$, $N = 100$, and $N = 200$ trials ran in 3.4, 7.0, 81.8, and 25,426.9 seconds, respectively. Unfortunately, in *ZGC_grapher.c* computing power became an issue, as for $N > 200$, the runtime of our programs became infeasible.

(a) $N = 50$ (b) $N = 75$ (a) $N = 100$ (b) $N = 200$

From these we can see that increasing N seems to correlate with a sharper transition from the region dominated by $G_{\#}$ to the region dominated by $Z_{\#}$ region in the $1.5 < k < 1.7$ range. Interestingly, the $1.4 < k < 1.5$ region remained relatively constant for all N trials despite being the beginning of the decaying region. Also, we can see a relatively small jump in $Z_{\#}$ at $k = 1.5$, which is likely due to a change in the behavior of V .

The other prominent feature of the ZGC graphs is the prevalence of $C_{\#}$ for $1.66 \lesssim k \lesssim 1.71$. Looking at the $N = 200$ case, $Z_{\#}$ levels off before $k \approx 1.66$ and returns to level after $k \approx 1.71$. Thus, this range where $C_{\#}$ is higher is likely some special behavior within the larger region, $k > 1.6$. Why this occurs and why this occurs here is open for further study.

Overall, this function seems to have a couple possible areas for further studies.

6. Sum of Divisors Function

In this section, we let $S(n) = \sigma(n) = \sum_{d|n} d$ which is to say $S(n)$ equals the sum of the positive divisors of positive integer n including 1 and n . Therefore, $S(n)$ is the Sum of Divisors function. Below is the resulting ZGC graph.

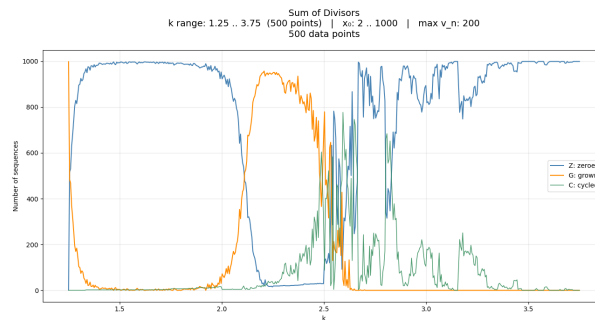


Figure 10: Time to compute: 9468.8s

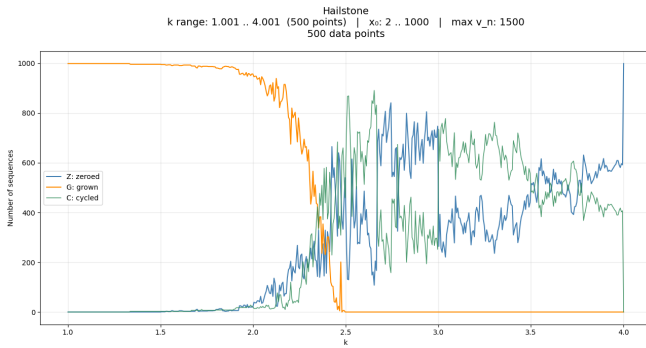
Within the non-trivial range of $1.25 < k < 3.75$, there are multiple regions with varying behavior. We see one dominated by $Z_{\#}$, another by $G_{\#}$, and another that is chaotic-looking. Furthermore, the transitions between these regions are sharp.

From the definition of the vector sequence it makes sense for there to exist a k value, k_0 such that for $k < k_0$, vector sequences predominately grow and for $k > k_0$, vector sequences to predominantly zero. The transition from predominantly growing, to zeroing, back to growing, and then back to zeroing, though, is a non-trivial behavior. What causes these extra transitions is an area for further study.

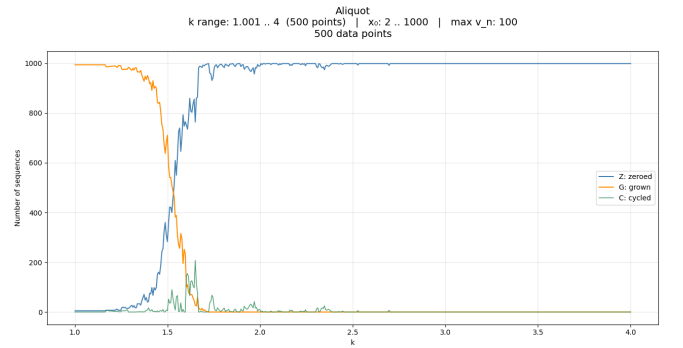
Due to computational limitations, we were unable to explore this $S(n)$ in more depth. We hypothesize that by varying either N and/or x_{end} this ZGC graph could show interesting behavior. This is also an area for further study.

7. Comparison

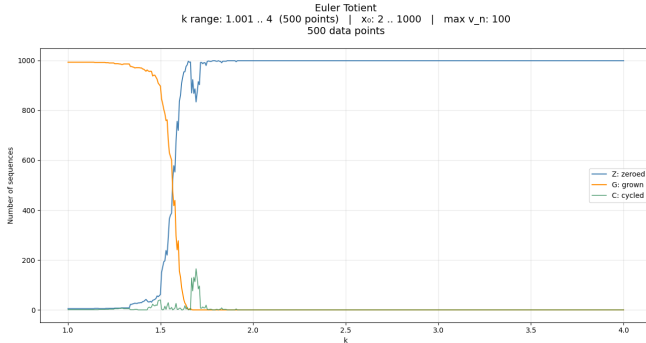
In this section, we compare the previous $S(n)$ cases by first showing the ZGC graphs for each. Again, for ease of comparison, we will use the same k range for each graph.



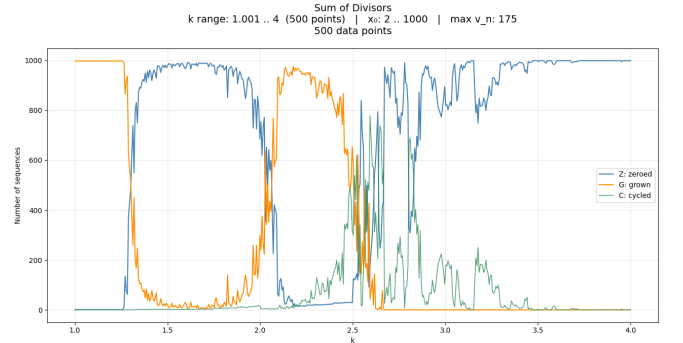
(a) General Hailstone Function ($q = 3$)



(b) Aliquot Function



(a) Euler-Totient Function



(b) Sum of Divisors Function

From the figures above, we see that the range $1 < k < 4$ includes at least one transition period for all functions studied. In other words, there existed an initial $G_{\#}$ dominated region and a final $Z_{\#}$ region. Like we stated in Section 6, this is intuitive from the role of k in the vector sequence. It is a decay parameter, so, as it increases, the amount of decay in the vector sequence also increases. It makes sense that at some point we can increase k enough to introduce enough decay that each vector sequence will zero. The value of k that does this is an area for further study. Also, why there seems to not exist outlier k -values beyond this where $C_{\#} > 0$ or $G_{\#} > 0$ is an area for further study. Furthermore, although the property is intuitive, it seems plausible that a set of properties of $S(n)$ may need to be true for this cutoff value of k to exist. Therefore, we conjecture the following.

Conjecture 4. For all $S(n)$ satisfying some general set of properties, considering any vector sequence, there exists an initial region of the ZGC graph such that $Z_{\#} = 0$ and $G_{\#} = \Sigma_{\#}$ and there exists a final region of the ZGC graph such that $Z_{\#} = \Sigma_{\#}$ and $G_{\#} = 0$.

Using much faster growing number theory functions for $S(n)$ would be a good test for this; we did not have the computational ability to do this.

Comparing the ZGC graphs of the Aliquot and Euler-Totient cases, we see similar features including only one transition, the one transition being relatively smooth and abrupt, an increase in $C_{\#}$ following the transition, minimal chaotic-looking behavior, and the $G_{\#}$ and $Z_{\#}$ lines appearing jagged during the transition. Differences appear primarily on a finer scale. Although the two graphs look like they could be the same $S(n)$ case with different N values, one differentiator is the k values the transitions occur at are different. It is not clear by the direct relationship $Aliquot(n) = s(n) = \sum_{d|n} (d - \phi(n))$, where $\phi(n)$ is the Euler-Totient function, that these cases should have such similar ZGC graphs; therefore, why they do is an area for further study.

Comparing the ZGC graphs of the General Hailstone and Sum of Divisors cases, we see similarities including chaotic-looking behavior, a later transition region, and overall more dynamism. Differences include the Sum of Divisor case having more transitions than the General Hailstone case, the General Hailstone case having more cycling vector sequences than the Sum of Divisors case, and the General Hailstone case having a discontinuous jump at $k = 4$ to $Z_{\#} = \Sigma_{\#}$ while the Sum of Divisors case has a smoother transition into this final region.

Note that no two transitions for different ZGC graphs occurred over the same k values. This could be a coincidence or this could be evidence that $S(n)$ has some bearing on where a ZGC graph transitions. In other words, $S(n)$ could have some bearing on where the vector sequence changes behavior. This is an area for further study.

In our studies there was only iteration of the ZGC graphs for the General Hailstone case. We expect that there are other interesting results for non-trivial $S(n)$ that have this property. This is an area for further study.

From our data, we claim that, in the context of a vector sequence, $S(n)$ is a chaotic parameter; small changes in the function result in large changes in the behavior of the vector sequence. This claim is an area for further study.

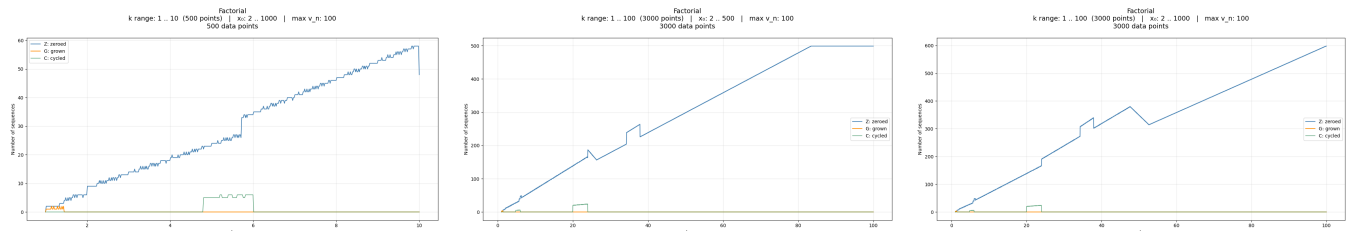
The goal of this paper has been to introduce the vector sequence allowing us to ask new questions about old functions. In addition to the previously mentioned areas for further study, other meta-questions about this object include why the drop in $G_{\#}$ from $\approx \Sigma_{\#}$ to 0 occurs around the k values it does, why any of the ZGC graphs demonstrate chaotic-looking behavior, and how to predict the impact of $S(n)$ on the vector sequence. All are areas for further exploration.

8. Additional Functions and Data

All of this data is from a program that is not published to the GitHub repository but are other examples of ZGC graphs for additional $S(n)$ choices. These are all areas for future study.

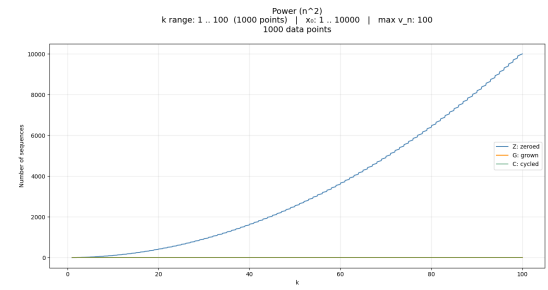
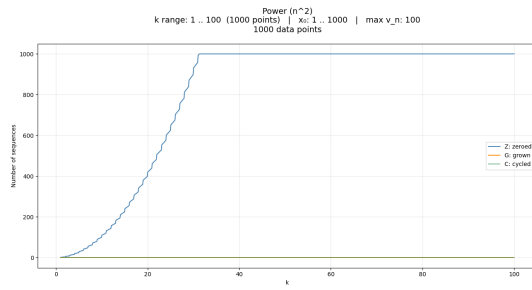
This program, unlike the one currently in the GitHub repository, does not include sequence in the graph that exceed the vector elements during the computation so that accuracy can be preserved. Thus, here, $G_{\#} + Z_{\#} + C_{\#}$ may or may not equal $\Sigma_{\#}$. In most cases it will not. Therefore, these graphs represent the most accurate data we have, as some of the growing sequences in the previous sections may actually zero or cycle. However, because of the size a number needs to be to exceed this limit, we can be fairly comfortable classifying sequences that reach this level as growing - if they do not actually grow then they should take a long time to zero or cycle. This is also why N was so important in the previous sections. N is still important for data in this section, but now reflects more intermediary vector sequences which have neither blown way up, nor zeroed. We are almost inclined to suggest that $G_{\#}$ does not give much information, while $C_{\#}$ and $Z_{\#}$ are the most useful.

8.1. Factorial Function

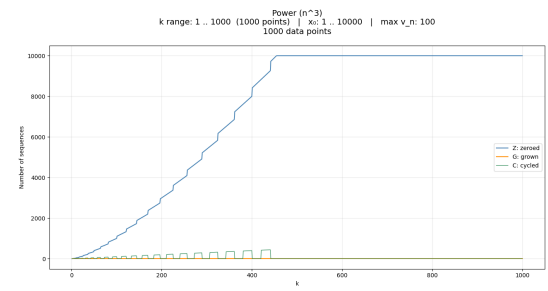
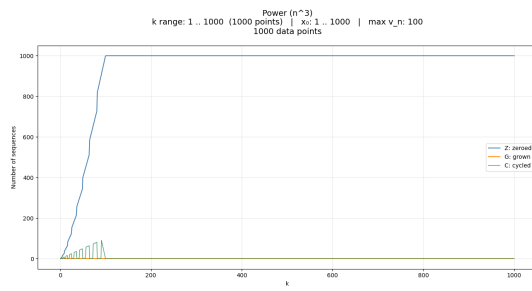


8.2. Power Functions

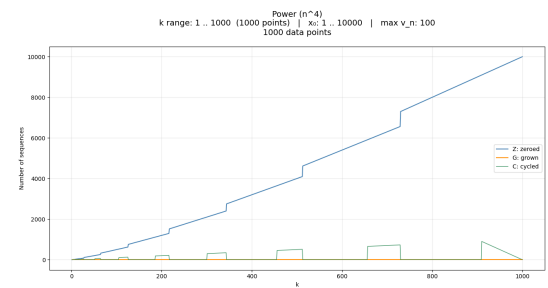
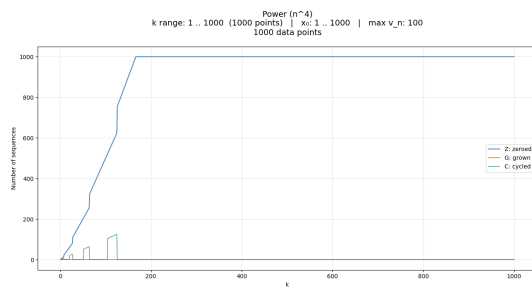
8.2.1. Quadratic



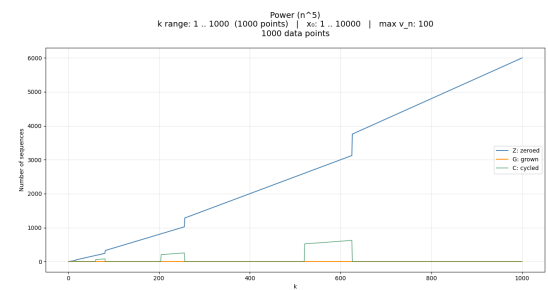
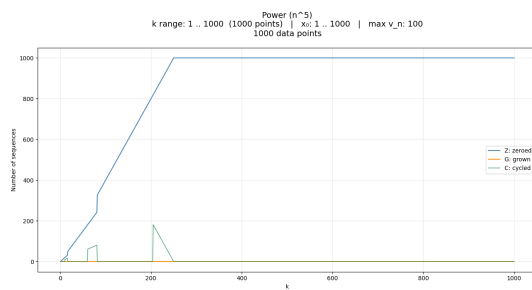
8.2.2. Cubic



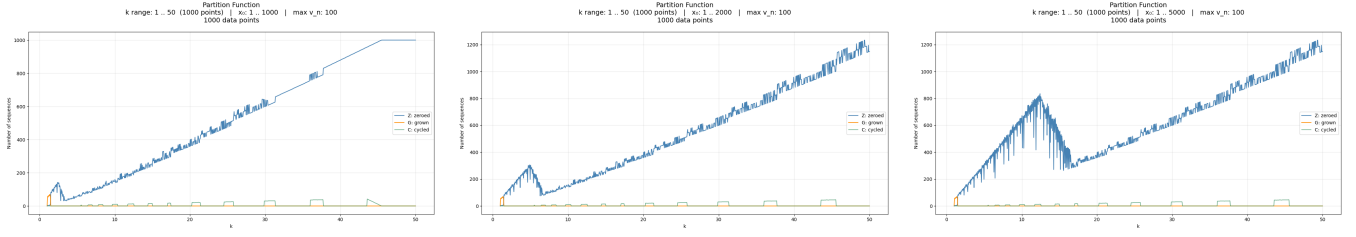
8.2.3. Quartic



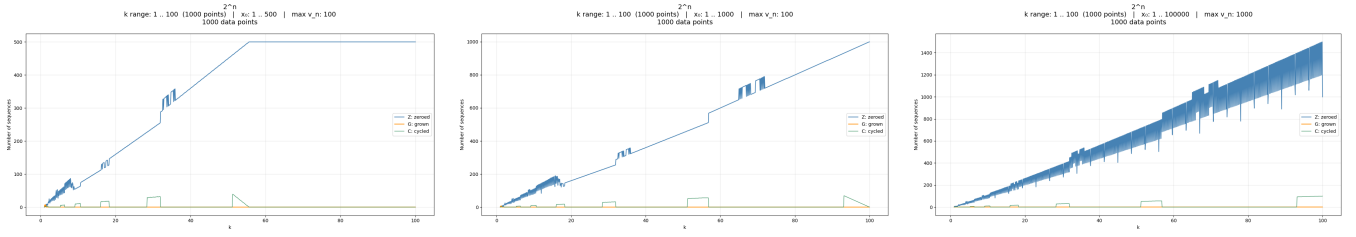
8.2.4. Quintic



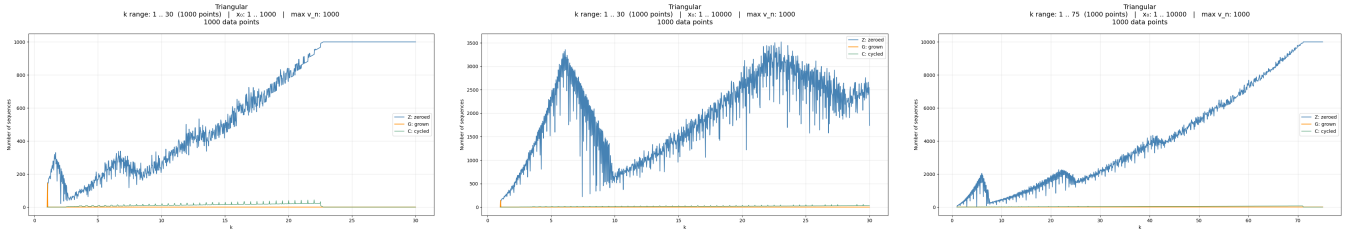
8.3. Partition Function



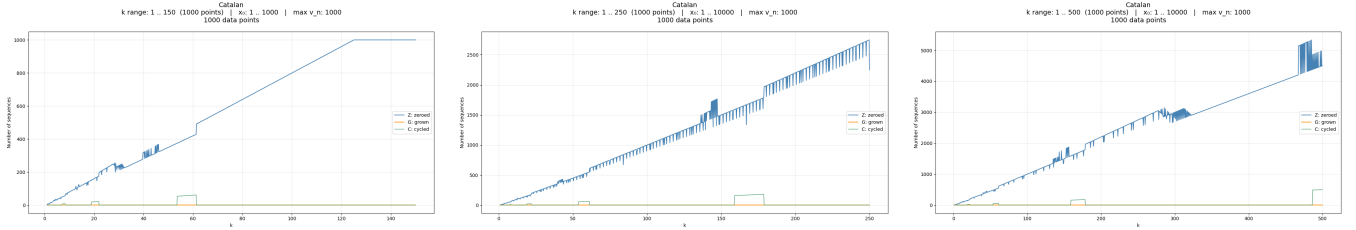
8.4. Exponential Function



8.5. Triangular Function



8.6. Catalan Function



9. Experimental Challenges and Limitations

For reference we used a 4 core, 8 thread 3.30 GHz 11th generation Intel i7-11370H CPU. Through testing, it quickly became evident that with this hardware, there were computational limits to our data collection. This resulted in our computational parameters ranging for different $S(n)$ cases, as seen in the titles of our figures. However, optimizations were made to achieve the speeds we did, but typically ranging from only 500 to 1000 data points per figure, we in no way claim that we have captured all useful data even in the k ranges our figures spanned. Considering the microstructure that exists for chaotic behavior and that our step size for k in our ZGC graphs was on the order of 10^{-6} , it seems undeniable that a lot of valuable information was missed.

The optimizations that were made to achieve our results were limited to optimizing the calculation of $S(n)$. This is best understood through a Big-O analysis of the vector sequence when creating a ZGC graph.

The general time-complexity for V depends on the number of data points (or k values), D , the value of x_{end} , the value of N , the length of the non-zero tail for each V , L , and the cost of each calculation of $S(n)$, C_S . It is trivial that

L grows linearly with N so we can assume an average L , $\bar{L}$, for all N . Also, for each data point (or k value), all x_0 up through x_{end} are checked, and for each x_0 , all v_n up through v_N are checked, and for each v_n , $S(n)$ is applied to the sum of the non-zero tail. Since the computation of $S(n)$ must come after the sum is calculated, these operations must be sequential. And since the sum is $O(\bar{L})$, the computation of $S(n)$ applied to this sum is $O(\bar{L} + C_S)$. Thus, $V = O(D \cdot x_{end} \cdot N \cdot (\bar{L} + C_S))$. Since D , x_{end} , N , and $\bar{L}$ are inherent parameters, they cannot be optimized, so we must focus on optimizing C_S .

To do this, we used C instead of Python for the data computations, reused buffers, implemented a non-zero tail representation, used hash-based cycle detection, allowed for variable-sized caching for the $S(n) = \sigma(n)$ and $S(n) = \phi(n)$ cases, and used CPU parallelization. For our parallelization, single-threaded processing was replaced by multi-threaded processing since parallelization is natural for the vector sequence algorithm. This alone resulted in a speedup of 2-5x for all tested $S(n)$ cases and k ranges with an experimentally optimal 4 threads. One time-complexity hit that we took was switching from floating point math to integer rational math. While this is slower, it preserves the accuracy we needed to accurately calculate the chaotic-looking behavior. Although the exact increase in speed still varies because it depends on $S(n)$ and k , we estimate that these optimizations resulted in a 10-100x speedup.

This said, there may be additional software optimizations we have overlooked; however, our recommendation would be to just optimize the hardware (i.e. buy a better CPU).

References

- [1] Leonard Eugene Dickson. *History of the Theory of Numbers, Vol. I: Divisibility and Primality*. Carnegie Institution of Washington, Washington, DC, 1919.
- [2] Leonhard Euler. Theoremata arithmetica nova methodo demonstrata. *Novi Commentarii Academiae Scientiarum Petropolitanae*, 8:74–104, 1763.
- [3] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008.
- [4] Jeffrey C. Lagarias. The $3x+1$ problem and its generalizations. *The American Mathematical Monthly*, 92(1):3–23, 1985.